

VARDHAMAN

COLLEGE OF ENGINEERING

A Course End Project Report on

Supply Chain Management Database using Oracle SQL

A9510 – DataBase Management System Laboratory

Submitted by

T. Raj Kumar	25881A0537
P. Shivaji Goud	25881A0554
T. Yuval Anand Reddy	25881A0565

Course Facilitator

Ms. Shaik Shahanaz

Assistant Professor

Department of Computer Science and Engineering

Vardhaman College of Engineering, Hyderabad

October 2026



Autonomous Institute, Affiliated to **JNTUH**,
Approved by **AICTE**, Accredited by **NAAC** with **A++** Grade
Kacharam, Shamshabad, Hyderabad – 501 218,
Telangana, India.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

This is to certify that the Course End Project titled **"Supply Chain Management Database using Oracle SQL"** is carried out by **"T. Raj Kumar"** – **"25881A0537"**, **"P. Shivaji Goud"** – **"25881A0554"** and **"T. Yuval Anand Reddy"** – **"25881A0565"** towards **A9510 – Database Management System Laboratory** course in partial fulfilment of the requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** during the Academic year 2026–27.

Course Facilitator	Academic Head	Head of the Department
Ms. Shaik Shahanaz	**Dr. R. Karthikeyan**	**Prof. Y. Vijayalata**
Assistant Professor, CSE	Professor, CSE	HoD, CSE

Contents

1 Introduction **5**

2 Literature Review **5**

3 Objectives **6**

4 Problem Analysis **6**
4.1 Problem Statement 6
4.2 Users and Assumptions 6
4.3 Functional Requirements 7
4.4 System Architecture (Block Diagram) 8
4.5 Tools and Technologies 8

5 ER Model **9**
5.1 Entities and Attributes 9
5.2 Relationships 9
5.3 Cardinality and Participation 9
5.4 ER Diagram 10

6 Relational Model **10**
6.1 Relational Schema 10
6.2 Primary Keys and Foreign Keys 11

7 Normalization **11**
7.1 Functional Dependencies 11
7.2 First Normal Form (1NF) 11
7.3 Second Normal Form (2NF) 11
7.4 Third Normal Form (3NF) 11

8 SQL DDL Commands **12**
8.1 Table Creation 12
8.2 Constraints 12
8.3 ALTER Operation 13

9 SQL DML Commands **13**
9.1 INSERT 13
9.2 UPDATE 14
9.3 DELETE 14
9.4 GRANT and REVOKE 14

10 SQL Queries **15**
10.1 Basic SELECT Queries 15
10.2 Joins 15

11 Transactions and Concurrency **15**
11.1 Transaction Management 15
11.2 COMMIT, ROLLBACK and SAVEPOINT 15
11.3 Concurrency Control 16

12 PL/SQL Implementation **17**
12.1 Functions 17
12.2 Cursors 17

12.3 Triggers 17

13 Indexing **18**

14 Execution and Results **19**
14.1 How to Run the Project 19
14.2 Execution Environment and Note on the Outputs 19
14.3 Representative Query Outputs 19

15 Testing **22**
15.1 Test Cases 22
15.2 Invalid Input Testing 23

16 Reports **24**
16.1 Order Value Report 24
16.2 Low-Stock Report 24
16.3 Order Details Report 24

17 Results and Discussion **25**

18 Conclusion **25**

19 Mapping POs and SDGs **26**
19.1 Program Outcomes (POs) Mapping 26

20 Future Scope **27**

21 Bibliography **27**

Abstract

Managing the purchasing and stock of a business involves a large amount of inter-related data: suppliers, products, purchase orders and shipments. Handling this data on paper or in separate spreadsheets leads to duplication, inconsistency, poor shipment visibility and inaccurate stock figures. This project designs and implements a Supply Chain Management Database (SCMD) as a relational database using Oracle SQL and PL/SQL.

The database is modelled with an Entity–Relationship (ER) diagram drawn in ERDPlus, containing four entities (Product, Supplier, Purchase Order and Shipment), and mapped to four relational tables that are normalized up to Boyce–Codd Normal Form (BCNF). Integrity is enforced with primary keys, foreign keys, NOT NULL, CHECK and DEFAULT constraints, and with two database triggers: one supplies the order date automatically and the other adds the ordered quantity to the product stock when a shipment is marked as delivered. The project demonstrates the core topics of a DBMS course: DDL (including ALTER TABLE), DML, a three-table join, a stored function that computes the value of an order, a cursor that reports low-stock products, access control with GRANT and REVOKE, and transaction control with COMMIT, SAVEPOINT and ROLLBACK. All code is kept deliberately simple so that it can be explained and reproduced during a viva voce.

Keywords: Database Management System, Oracle SQL, PL/SQL, supply chain management, normalization, BCNF, ER model, triggers, cursors, functions, transactions, ACID.

1 Introduction

Supply chain management is the coordination of the movement of goods and the related information from suppliers, through storage locations, to the point of use. Even a small business must keep track of who supplies what, which orders have been placed, which shipments are on the road, what has been delivered and how much stock of each product is available. When this information is maintained manually or across disconnected spreadsheets, the same fact (for example a supplier's contact details) is stored many times, updates are easy to miss, and producing a simple report such as "which products are running low" becomes a time-consuming and error-prone task.

A Database Management System (DBMS) removes these problems by storing data once, in a structured form, and by enforcing rules that keep the data consistent. This project applies the relational model to build a Supply Chain Management Database (SCMD) on Oracle Database. The system stores information about suppliers, products, purchase orders and shipments, and exposes it through queries, a stored function and a cursor-based report.

Beyond simply storing data, the project is intended as a practical demonstration of the topics covered in a DBMS course. It uses Data Definition Language (DDL) with integrity constraints, Data Manipulation Language (DML), queries with joins, and PL/SQL program units — a function, a cursor and triggers — together with explicit transaction control and access control. The design is normalized to eliminate redundancy.

2 Literature Review

The relational model, on which this system is built, was introduced by E. F. Codd [7] and is described in detail by Date [4]. Chen's Entity–Relationship model [8] provides the standard way of identifying entities and relationships before they are converted into tables. The standard treatment of relational design, normalization and query processing follows Elmasri and Navathe [1] and Silberschatz, Korth and Sudarshan [2]; the theory of functional dependencies and the normal forms (1NF through BCNF) used in Section 7 is taken from these texts. Ramakrishnan and Gehrke [3] provide the treatment of transaction management, the ACID properties, concurrency control and recovery that underpin Section 11.

The practical use of Oracle's SQL dialect and PL/SQL procedural extension follows the official Oracle documentation: the SQL Language Reference [5] and the PL/SQL Language Reference [6]. Supply chain literature [9] emphasizes that coordinated information flow between suppliers, warehouses and transport providers improves responsiveness and reduces cost, which motivates keeping this information in one consistent store.

Enterprise resource planning (ERP) suites cover supply chain functions commercially, but they are large, closed systems whose internal data design is not the subject of study. Spreadsheets, on the other hand, are easy to start with but are difficult to keep consistent as the number of suppliers, orders and shipments grows. In contrast, the present work concentrates on demonstrating sound database design and the main DBMS features — constraints, normalization, joins, triggers, cursors, functions, access control and transactions — within one coherent, reproducible Oracle schema.

3 Objectives

- To design an ER model for a supply chain domain and map it to a relational schema normalized to BCNF.
- To implement the schema in Oracle using DDL with primary key, foreign key, NOT NULL, CHECK and DEFAULT constraints, and to modify it with ALTER TABLE.
- To populate the database with realistic sample data using INSERT, UPDATE and DELETE.
- To demonstrate SQL querying with SELECT statements and a three-table join.
- To implement PL/SQL program units: a function, a cursor and triggers (automatic stock update and default date).
- To illustrate transactions and the ACID properties using SAVEPOINT, COMMIT and ROLLBACK, and access control using GRANT and REVOKE.
- To capture the commands and outputs in SQL*Plus so that they can be reproduced for a viva voce.

4 Problem Analysis

The system was developed in the classic database-design sequence: requirements analysis, conceptual design (ER model), logical design (relational schema and normalization), and implementation and testing in Oracle SQL*Plus. The subsections below describe the problem, the users, the requirements, the architecture and the tools used.

4.1 Problem Statement

A business may keep supplier, purchase, shipment and stock information in separate files. This creates duplicate records, makes it difficult to determine the status of an order, and delays the identification of low-stock products. A central relational database is required to link these records accurately and to provide useful reports.

A relational database with linked tables and SQL operations is therefore developed. It allows authorized users to register suppliers and products, create purchase orders, track shipments, keep the stock of every product up to date when goods are delivered, and list the products that are running low.

4.2 Users and Assumptions

The users of the system and their responsibilities are given in Table 1.

The system makes the following assumptions:

- One purchase order is placed with one supplier; a supplier can receive many purchase orders.
- In this simplified design a purchase order is for one product and one quantity; a product can appear in many purchase orders.
- A purchase order may have one or more shipment records if goods are dispatched separately.

Table 1: System Users and their Responsibilities

User	**Typical responsibilities**
Purchase officer	Maintains suppliers and creates purchase orders.
Warehouse operator	Updates product stock quantities.
Logistics staff	Enters shipment dates and shipment status.
Manager	Reviews order values and low-stock reports.
Database administrator	Manages access, backups, integrity and performance.

- A shipment has the status `In Transit` (default) or `Delivered`.
- The value of an order is computed from the current unit price of the ordered product; the price is stored only in the Product table.
- The stock of a product increases by the ordered quantity when a shipment of that order is updated to `Delivered`.
- If no order date is supplied, the system date is used.

4.3 Functional Requirements

The functional requirements that the database must satisfy are listed in Table 2.

Table 2: Functional Requirements

ID	**Requirement**	**Description**
FR-01	Supplier management	Add, view and update supplier records (name, phone and address).
FR-02	Product management	Maintain product names, unit prices and stock quantities.
FR-03	Purchase processing	Create purchase orders for a supplier, product and quantity; the order date is set automatically when omitted.
FR-04	Shipment tracking	Record the ship date and status of the shipments of an order.
FR-05	Stock update on delivery	Add the ordered quantity to the product stock when a shipment is marked as delivered.
FR-06	Order valuation	Compute the value of an order from its quantity and the unit price of the product.
FR-07	Low-stock monitoring	List the products whose stock is below the reorder threshold.
FR-08	Access control	Grant and revoke read access to the product data.
FR-09	Transaction control	Group changes into transactions using COMMIT, SAVEPOINT and ROLLBACK.

The non-functional requirements are data integrity (related records must reference valid parent records), usability (understandable table and column names), maintainability (new products and suppliers can be added easily), security (read access can be granted and revoked), reliability (transactions for multi-step updates) and portability (common relational concepts, with minor syntax changes between Oracle and MySQL).

4.4 System Architecture (Block Diagram)

The SCMD follows the standard client–server architecture of an Oracle database. A user (purchase officer, warehouse operator or administrator) issues SQL and PL/SQL statements through the SQL*Plus client. The Oracle server parses and optimizes the SQL, executes procedural logic in the PL/SQL engine, and coordinates concurrent access through the transaction and lock manager, before reading from and writing to the database tables.

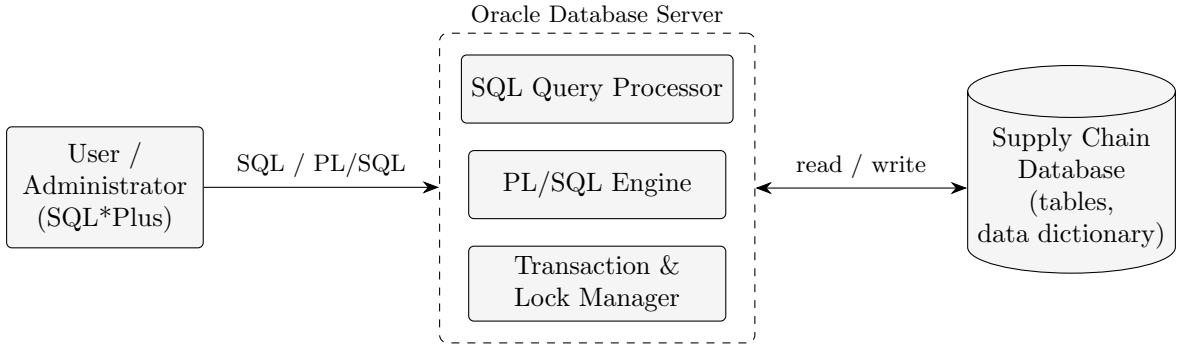


Figure 1: Block diagram of the Supply Chain Management Database

Functionally, the system is organized into the following modules:

- **Procurement module** — maintains suppliers, products and purchase orders.
- **Logistics module** — maintains shipments (ship date and status).
- **Inventory module** — keeps the stock quantity of every product, updated automatically when a shipment is delivered.
- **Reporting module** — provides the order-value function, the low-stock cursor and join queries.
- **Integrity module** — constraints, triggers and transactions that keep the data valid and consistent.

4.5 Tools and Technologies

- **Oracle Database** — the relational database server.
- **SQL*Plus** — the command-line client used to run the scripts and capture the output.
- **SQL and PL/SQL** — for data definition, querying and procedural logic.
- **ERDPlus** — for drawing the ER diagram.
- LaTeX / Overleaf — for preparing this report; code is typeset with the listings package.

5 ER Model

The Entity–Relationship (ER) model is used to represent the structure of the Supply Chain Management Database and the relationships between its different entities. The conceptual model contains four entities: Product, Supplier, Purchase Order and Shipment. Each entity contains attributes that describe its important information and is identified using a primary key.

5.1 Entities and Attributes

Product, Supplier, Purchase Order and Shipment are all strong entities. Purchase Order links a supplier with a product, so it carries the foreign keys `supplier_id` and `product_id`; Shipment carries the foreign key `order_id` of the order it belongs to. These foreign-key attributes are also shown inside the ERDPlus diagram.

Table 3: Entities and Attributes of the System

Entity	**Attributes**	**Keys**
Product	product_id, product_name, unit_price, stock_quantity	PK: product_id
Supplier	supplier_id, supplier_name, phone, address	PK: supplier_id
Purchase Order	order_id, supplier_id, product_id, quantity, order_date	PK: order_id FK: supplier_id, product_id
Shipment	shipment_id, order_id, ship_date, status	PK: shipment_id FK: order_id

The attribute `address` of Supplier was added after the table was created, using ALTER TABLE (Section 8.3).

5.2 Relationships

A supplier can have many purchase orders, and each order belongs to one supplier (*has orders*). A product can be ordered in many purchase orders, and each order is for one product (*ordered by*). A purchase order can be split across multiple shipments, and each shipment belongs to exactly one order (*has shipments*).

5.3 Cardinality and Participation

Table 4: Relationships, Cardinality and Participation

Relationship	**Entities**	**Cardinality**	**Participation**
has orders	Supplier – Purchase Order	1 : N	Purchase Order: total, Supplier: partial
ordered by	Product – Purchase Order	1 : N	Purchase Order: total, Product: partial
has shipments	Purchase Order – Shipment	1 : N	Shipment: total, Purchase Order: partial

5.4 ER Diagram

The ER diagram represents the entities, attributes, relationships and cardinalities of the Supply Chain Management Database. Rectangles are entities, ellipses are attributes (key attributes are underlined), diamonds are relationships and the crow's-foot symbol marks the "many" side. It provides a graphical representation of the database structure and serves as the basis for converting the ER model into relational tables.

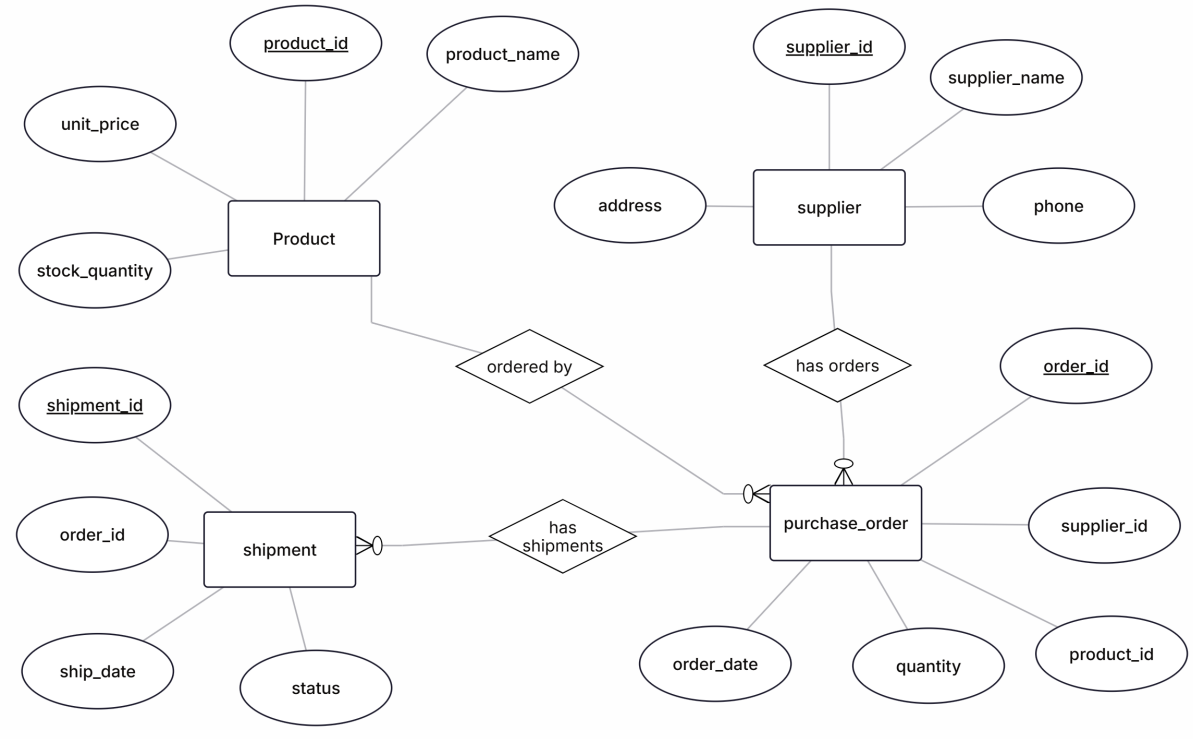


Figure 2: ER Diagram of the Supply Chain Management Database

6 Relational Model

The relational model represents the data of the Supply Chain Management Database in the form of tables. Each table contains related attributes, with primary keys used to uniquely identify records and foreign keys used to establish relationships between tables.

6.1 Relational Schema

The ER model was mapped to the following relations. Primary keys are underlined and foreign keys are listed in Table 5.

- Product(product_id, product_name, unit_price, stock_quantity)
- Supplier(supplier_id, supplier_name, phone, address)
- Purchase_Order(order_id, supplier_id, product_id, quantity, order_date)
- Shipment(shipment_id, order_id, ship_date, status)

6.2 Primary Keys and Foreign Keys

Each table has a primary key that uniquely identifies every record. Foreign keys are used to connect related tables: an order must refer to an existing supplier and an existing product, and a shipment must refer to an existing order. Because Oracle checks these rules, a child record cannot be created without its parent, and a parent that still has child records cannot be deleted.

Table 5: Foreign Keys in the Schema

Table	**Foreign Key**	**References**
Purchase_Order	supplier_id	Supplier(supplier_id)
Purchase_Order	product_id	Product(product_id)
Shipment	order_id	Purchase_Order(order_id)

7 Normalization

Normalization is the process of organizing data in a database to reduce redundancy and avoid insertion, deletion and update anomalies. The Supply Chain Management Database is designed to be normalized up to Third Normal Form (3NF), and every table also satisfies BCNF.

7.1 Functional Dependencies

Functional dependencies describe how one attribute (or set of attributes) uniquely determines others. In the proposed system, the following functional dependencies are identified:

- product_id $\rightarrow$ product_name, unit_price, stock_quantity
- supplier_id $\rightarrow$ supplier_name, phone, address
- order_id $\rightarrow$ supplier_id, product_id, quantity, order_date
- shipment_id $\rightarrow$ order_id, ship_date, status

7.2 First Normal Form (1NF)

Every attribute holds a single atomic value and there are no repeating groups. Each order is a single row with one product and one quantity, and the several shipments of an order are stored as separate rows of the Shipment relation rather than in a repeating column.

7.3 Second Normal Form (2NF)

There is no partial dependency of a non-key attribute on part of a candidate key. All four relations have single-column primary keys, so 2NF holds trivially.

7.4 Third Normal Form (3NF)

There is no transitive dependency among non-key attributes. For example, supplier details are stored once in Supplier and referenced by supplier_id from Purchase_Order, and product details are stored once in Product and referenced by product_id. The unit price is kept only in Product, so the value of an order is computed from it and is never stored as a redundant copy.

For every non-trivial functional dependency $X \to Y$, X is a superkey. All determinants in the schema are the primary keys (supplier_id, product_id, order_id and shipment_id). Hence every relation is also in Boyce–Codd Normal Form (BCNF).

8 SQL DDL Commands

Data Definition Language (DDL) commands are used to create and modify the structure of database tables. In the Supply Chain Management Database, DDL commands are used to create tables and define constraints such as primary keys, foreign keys, NOT NULL, CHECK and DEFAULT. The statements below use Oracle SQL syntax and were executed in SQL*Plus; each returned the message `Table created.`

8.1 Table Creation

The tables are created in the order of dependency, so that parent tables (Product and Supplier) exist before the tables that reference them (Purchase_Order and then Shipment).

```
CREATE TABLE product (
  product_id      NUMBER         PRIMARY KEY,
  product_name    VARCHAR2(50)   NOT NULL,
  unit_price      NUMBER(8,2)    NOT NULL CHECK (unit_price >= 0),
  stock_quantity  NUMBER         DEFAULT 0 CHECK (stock_quantity >= 0)
);

CREATE TABLE supplier (
  supplier_id    NUMBER         PRIMARY KEY,
  supplier_name  VARCHAR2(50)   NOT NULL,
  phone          VARCHAR2(15)
);

CREATE TABLE purchase_order (
  order_id     NUMBER  PRIMARY KEY,
  supplier_id  NUMBER  NOT NULL REFERENCES supplier(supplier_id),
  product_id   NUMBER  NOT NULL REFERENCES product(product_id),
  quantity     NUMBER  NOT NULL CHECK (quantity > 0),
  order_date   DATE    NOT NULL
);

CREATE TABLE shipment (
  shipment_id  NUMBER       PRIMARY KEY,
  order_id     NUMBER       NOT NULL REFERENCES purchase_order(order_id),
  ship_date    DATE,
  status       VARCHAR2(15) DEFAULT 'In Transit'
               CHECK (status IN ('In Transit', 'Delivered'))
);
```

8.2 Constraints

- PRIMARY KEY uniquely identifies each record (and makes it NOT NULL).
- FOREIGN KEY (REFERENCES) ensures that referenced suppliers, products and orders exist.
- NOT NULL is applied to required values such as names, prices, quantities and order dates.
- CHECK rejects a negative unit price or stock quantity, an order quantity that is zero or negative, and a shipment status other than `In Transit` or `Delivered`.
- DEFAULT supplies a stock of 0 for a new product and the status `In Transit` for a new shipment.

8.3 ALTER Operation

The ALTER command is used to change the structure of an existing table. An address column was added to Supplier and the width of the phone column was increased from 15 to 20 characters.

```
ALTER TABLE supplier ADD address VARCHAR2(100);
ALTER TABLE supplier MODIFY phone VARCHAR2(20);
```

Both statements returned `Table altered.`

9 SQL DML Commands

Data Manipulation Language (DML) commands are used to insert, modify and delete records in database tables. In the Supply Chain Management Database, DML commands are used to manage product, supplier, order and shipment information.

9.1 INSERT

The INSERT command is used to add new records to a table. The database is populated with four products, three suppliers, three purchase orders and two shipments; each statement returned `1 row created.` The data was then made permanent with COMMIT.

```
INSERT INTO supplier (supplier_id, supplier_name, phone)
  VALUES (1, 'Apex Components', '9000000001');
INSERT INTO supplier (supplier_id, supplier_name, phone)
  VALUES (2, 'Nova Packaging', '9000000002');
INSERT INTO supplier (supplier_id, supplier_name, phone)
  VALUES (3, 'Vertex Industrial', '9000000003');

INSERT INTO product VALUES (1, 'Microcontroller Board', 850, 8);
INSERT INTO product VALUES (2, 'Protective Carton',      35, 150);
INSERT INTO product VALUES (3, 'Battery', 420, 4);
INSERT INTO product VALUES (4, 'solar panels', 2000, 50);

INSERT INTO purchase_order VALUES (1, 1, 1, 20,  DATE '2026-10-01');
INSERT INTO purchase_order VALUES (2, 2, 2, 100, DATE '2026-10-03');
INSERT INTO purchase_order VALUES (3, 1, 3, 10,  DATE '2026-10-05');

INSERT INTO shipment VALUES (1, 1, DATE '2026-10-04', 'In Transit');
INSERT INTO shipment VALUES (2, 3, DATE '2026-10-06', 'Delivered');

COMMIT;
```

Representative rows of the loaded data are shown below. Supplier 3 has no orders and product 4 has not been ordered, which makes it possible to see rows that do not take part in a join.

Table 6: Sample Rows from the Product Relation

product_id	product_name	unit_price	stock_quantity
1	Microcontroller Board	850	8
2	Protective Carton	35	150
3	Battery	420	4
4	solar panels	2000	50

Table 7: Sample Rows from the Supplier Relation

supplier_id	supplier_name	phone	address
1	Apex Components	9000000001	NULL
2	Nova Packaging	9000000002	NULL
3	Vertex Industrial	9000000003	NULL

9.2 UPDATE

The UPDATE command is used to modify existing records in a table. The first statement changes the unit price of product 1 from 850 to 900; because the price is stored only in Product, the value of every order for this product changes with it (Section 12.1). The second statement marks shipment 1 as delivered once the goods arrive; after the triggers of Section 12.3 were created, this update also increased the stock of the ordered product.

```
UPDATE product SET unit_price = 900 WHERE product_id = 1;
UPDATE shipment SET status = 'Delivered' WHERE shipment_id = 1;
```

Each statement returned `1 row updated.`

9.3 DELETE

The DELETE command is used to remove records from a table. A test supplier was inserted and then deleted; the change was saved with COMMIT. A supplier that is referenced by purchase orders cannot be deleted because of the foreign key constraint, which protects data integrity.

```
INSERT INTO supplier (supplier_id, supplier_name) VALUES (9, 'Test Supplier');
DELETE FROM supplier WHERE supplier_id = 9;
COMMIT;
```

9.4 GRANT and REVOKE

Data Control Language (DCL) commands are used to give and withdraw privileges. Read access to the Product table was granted to all users and then withdrawn again.

```
GRANT SELECT ON product TO PUBLIC;
REVOKE SELECT ON product FROM PUBLIC;
```

The commands returned `Grant succeeded.` and `Revoke succeeded.`

10 SQL Queries

SQL queries are used to retrieve and analyze data stored in the Supply Chain Management Database. The system uses SELECT statements with filtering and a join of three tables to obtain useful information.

10.1 Basic SELECT Queries

Projection and selection with a WHERE clause, and a query of all columns of a table.

```
-- Stock of product 1
SELECT product_name, stock_quantity FROM product WHERE product_id = 1;

-- Order date of order 4
SELECT order_id, order_date FROM purchase_order WHERE order_id = 4;

-- All suppliers
SELECT * FROM supplier;
```

10.2 Joins

A join retrieves related information from two or more tables. The query below combines Purchase_Order with Supplier and Product to show who supplies each ordered product and in what quantity.

```
SELECT o.order_id, s.supplier_name, p.product_name, o.quantity
FROM purchase_order o
JOIN supplier s ON s.supplier_id = o.supplier_id
JOIN product  p ON p.product_id  = o.product_id;
```

11 Transactions and Concurrency

Transactions are a sequence of database operations performed as a single logical unit. In the Supply Chain Management Database, transactions help ensure that changes to suppliers, products and orders are completed correctly and consistently.

11.1 Transaction Management

ACID. Atomicity means a transaction's statements all take effect or none do; the SAVEPOINT/ROLLBACK demonstration shows this. Consistency means a transaction moves the database between valid states; the constraints and triggers guarantee this. Isolation means concurrent transactions do not see each other's intermediate results; Oracle provides this through locking and its default READ COMMITTED isolation. Durability means once a transaction is COMMITted its effects survive failures, which Oracle guarantees through redo logs.

11.2 COMMIT, ROLLBACK and SAVEPOINT

COMMIT permanently saves the changes made by a transaction. ROLLBACK cancels the changes made since the beginning of the transaction or since the last SAVEPOINT. SAVEPOINT creates a point within a transaction to which the transaction can be rolled back. In the demonstration below, two suppliers are inserted with a savepoint between them; rolling back to the savepoint removes only the second insert, so one temporary supplier remains pending until the final ROLLBACK removes it as well.

```
INSERT INTO supplier (supplier_id, supplier_name) VALUES (4, 'Temp One');
savepoint sp1;
INSERT INTO supplier (supplier_id, supplier_name) VALUES (5, 'Temp Two');
rollback to sp1;
SELECT COUNT(*) AS temp_suppliers FROM supplier WHERE supplier_id > 3;
ROLLBACK;
```

COMMIT is used in the opposite way: after the price update, the insertion of supplier 9 and its deletion (Section 9), the transaction was ended with COMMIT so that the changes became permanent.

11.3 Concurrency Control

Oracle uses locks to serialize conflicting operations. A row that is being updated is locked until the transaction ends, and SELECT ... FOR UPDATE can be used to lock rows explicitly; row-level locking allows high concurrency because readers never block writers. This matters for stock: two users updating the same product at the same time must not overwrite each other's changes. The isolation level (READ COMMITTED by default, or SERIALIZABLE) controls phenomena such as non-repeatable reads and phantoms. A multi-session demonstration of locking was outside the scope of this project and is listed under future work.

12 PL/SQL Implementation

PL/SQL is a procedural extension of SQL that allows programming constructs such as variables, conditions, loops, functions, cursors and triggers. In the Supply Chain Management Database, PL/SQL is used to compute order values, report low stock and automate the date and stock handling. The programs are stored in script files (`cep1.sql`, `cep2c.sql`, `cep3t.sql` and `cep4t.sql`) and are run from SQL*Plus with `set serveroutput on`.

12.1 Functions

A function is a stored PL/SQL program that accepts parameters and returns a value. The function `get_order_value` returns the value of an order (quantity multiplied by the current unit price of the ordered product). Because it is a function, it can be used inside a SELECT statement.

```
CREATE OR REPLACE FUNCTION get_order_value (p_order_id NUMBER)
RETURN NUMBER IS
  v_value NUMBER;
BEGIN
  SELECT o.quantity * p.unit_price
  INTO   v_value
  FROM   purchase_order o
  JOIN   product p ON p.product_id = o.product_id
  WHERE  o.order_id = p_order_id;
  RETURN v_value;
END;
/

SELECT order_id, get_order_value(order_id) AS order_value
FROM purchase_order;
```

12.2 Cursors

A cursor is used to process the rows returned by a query one at a time. The script `cep2c.sql` declares an explicit cursor, opens it, fetches the rows in a loop until `c_low%NOTFOUND` becomes true, closes it, and prints a message for every product whose stock is below 10 units.

```
DECLARE
  CURSOR c_low IS
    SELECT product_name, stock_quantity FROM product
    WHERE stock_quantity < 10 ORDER BY product_id;
  v_name  product.product_name%TYPE;
  v_stock product.stock_quantity%TYPE;
BEGIN
  OPEN c_low;
  LOOP
    FETCH c_low INTO v_name, v_stock;
    EXIT WHEN c_low%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(v_name || ' has only ' || v_stock || ' left');
  END LOOP;
  CLOSE c_low;
END;
/
```

12.3 Triggers

A trigger is a PL/SQL program that is automatically executed when a specified database event occurs. Two triggers are defined. The trigger `trg_update_stock` (`cep3t.sql`) is an AFTER UPDATE trigger on Shipment: when the shipment status changes to `Delivered`, it adds the ordered quantity to the stock of the ordered product. The trigger `trg_default_date` (`cep4t.sql`)

is a BEFORE INSERT trigger on Purchase_Order that fills in the order date with the system date when none is supplied. Both returned the message `Trigger created.`

```
CREATE OR REPLACE TRIGGER trg_update_stock
AFTER UPDATE OF status ON shipment
FOR EACH ROW
WHEN (NEW.status = 'Delivered' AND OLD.status <> 'Delivered')
BEGIN
  UPDATE product
  SET    stock_quantity = stock_quantity +
           (SELECT quantity FROM purchase_order WHERE order_id = :NEW.order_id)
  WHERE  product_id =
           (SELECT product_id FROM purchase_order WHERE order_id = :NEW.order_id);
END;
/

CREATE OR REPLACE TRIGGER trg_default_date
BEFORE INSERT ON purchase_order
FOR EACH ROW
BEGIN
  IF :NEW.order_date IS NULL THEN
    :NEW.order_date := SYSDATE;
  END IF;
END;
/
```

13 Indexing

Indexing is used to improve the speed of data retrieval by providing an efficient way to locate records. Oracle automatically creates a unique index for every PRIMARY KEY constraint, so the columns product_id, supplier_id, order_id and shipment_id are already indexed. Oracle does not create indexes for foreign keys; no additional indexes were created in this project because the sample data is small. As the data grows, B-tree indexes on the foreign-key columns used in joins (supplier_id, product_id and order_id) and on the shipment status column would allow the optimizer to avoid full table scans.

14 Execution and Results

14.1 How to Run the Project

1. Start SQL*Plus and connect to the Oracle database, for example: `sqlplus username/password@orcl`
2. Create the four tables with the statements of Section 8 (product, supplier, purchase_order, shipment) and apply the ALTER TABLE statements.
3. Insert the sample data of Section 9 and issue COMMIT.
4. Run `set serveroutput on;` and then the script files in the order `@cep1.sql`, `@cep2c.sql`, `@cep3t.sql` and `@cep4t.sql`.
5. Execute the UPDATE, SELECT and transaction statements of Sections 9–11 and compare the results with those in Section 14.3.

14.2 Execution Environment and Note on the Outputs

The statements were run from SQL*Plus on Oracle Database, and the outputs reproduced in this report are the results of that session on the sample data of Section 9.1. The results depend on the order of execution: the low-stock cursor reports the stock before any shipment was delivered (8 microcontroller boards and 4 batteries), the order values reflect the unit price of 900 set by the UPDATE statement, and the stock of product 1 rises from 8 to 28 only after shipment 1 is marked as delivered with the stock trigger in place. The order date shown for order 4 is the date on which the session was run.

14.3 Representative Query Outputs

The console blocks below are the results of the queries and demonstrations of this report.

All suppliers (`SELECT * FROM supplier;`), after the ALTER TABLE statements:

```
SUPPLIER_ID SUPPLIER_NAME                                      PHONE
----------- -------------------------------------------------- --------------------
ADDRESS
--------------------------------------------------------------------------------
          1 Apex Components                                    9000000001

          2 Nova Packaging                                     9000000002

          3 Vertex Industrial                                  9000000003
```

Join of Purchase_Order, Supplier and Product:

```
  ORDER_ID SUPPLIER_NAME
---------- --------------------------------------------------
PRODUCT_NAME                                       QUANTITY
-------------------------------------------------- ----------
         1 Apex Components
Microcontroller Board                                      20

         2 Nova Packaging
Protective Carton                                         100

         3 Apex Components
Battery                                                    10
```

Function `get_order_value` (after the unit price of product 1 was set to 900):

```
SQL> set serveroutput on;
SQL> @"cep1.sql";

Function created.

SQL> SELECT order_id, get_order_value(order_id) AS order_value
  2  FROM purchase_order;

  ORDER_ID ORDER_VALUE
---------- -----------
         1       18000
         2        3500
         3        4200
```

Cursor (low-stock report, before any delivery was recorded):

```
SQL> @"cep2c.sql";
Microcontroller Board has only 8 left
Battery has only 4 left
```

Triggers, shipment update and order date:

```
SQL> @"cep3t.sql";

Trigger created.

SQL> @"cep4t.sql";

Trigger created.

SQL> UPDATE shipment SET status = 'Delivered' WHERE shipment_id = 1;

1 row updated.

SQL> SELECT product_name, stock_quantity FROM product WHERE product_id = 1;

PRODUCT_NAME                                STOCK_QUANTITY
------------------------------------------- --------------
Microcontroller Board                                 28

SQL> INSERT INTO purchase_order (order_id, supplier_id, product_id, quantity)
  2    VALUES (4, 2, 1, 5);

1 row created.

SQL> SELECT order_id, order_date FROM purchase_order WHERE order_id = 4;

  ORDER_ID ORDER_DAT
---------- ---------
         4 14-OCT-26
```

Transaction demonstration (SAVEPOINT and ROLLBACK TO):

```
SQL> INSERT INTO supplier (supplier_id, supplier_name) VALUES (4, 'Temp One');

1 row created.

SQL> savepoint sp1;

Savepoint created.

SQL> INSERT INTO supplier (supplier_id, supplier_name) VALUES (5, 'Temp Two');

1 row created.

SQL> rollback to sp1;

Rollback complete.

SQL> SELECT COUNT(*) AS temp_suppliers FROM supplier WHERE supplier_id > 3;

TEMP_SUPPLIERS
--------------
             1

SQL> ROLLBACK;
```

COMMIT, GRANT and REVOKE:

```
SQL> COMMIT;

Commit complete.

SQL> GRANT SELECT ON product TO PUBLIC;

Grant succeeded.

SQL> REVOKE SELECT ON product FROM PUBLIC;

Revoke succeeded.
```

15 Testing

Testing is performed to verify that the Supply Chain Management Database works correctly and produces the expected results. The tests check the integrity constraints, the DML statements, the triggers, the function, the cursor, the transaction control and the access control.

15.1 Test Cases

Table 8: Test Cases for the Supply Chain Management Database

Test ID	**Test Case**	**Expected Result**
T1	Insert an order without an order date	Date set to the system date (14-OCT-26) by the trigger
T2	Mark shipment 1 as delivered	Stock of product 1 rises from 8 to 28 (+20 ordered)
T3	Insert a supplier, SAVEPOINT, insert a second supplier, ROLLBACK TO the savepoint	Only the first supplier remains (count = 1)
T4	Run the low-stock cursor	Microcontroller Board (8) and Battery (4) listed
T5	Set the price of product 1 to 900 and call get_order_value	Order 1 = 18000 (20 × 900)
T6	Insert and delete supplier 9, then COMMIT	1 row created, 1 row deleted, commit complete
T7	GRANT and REVOKE SELECT on product	Grant succeeded, Revoke succeeded
T8	Join orders with suppliers and products	Three rows, one per order
T9	Insert an order for supplier 99 (no such supplier)	Foreign key violation (ORA-02291)
T10	Insert a product with unit price −5	CHECK violation (ORA-02290)
T11	Insert an order with quantity 0	CHECK violation (ORA-02290)
T12	Insert a shipment with status ‘Lost’	CHECK violation (ORA-02290)
T13	Delete supplier 1, which still has orders	Child record found (ORA-02292)
T14	Insert a supplier with an existing supplier_id	UNIQUE violation (ORA-00001)

15.2 Invalid Input Testing

Tests T9–T14 are designed to fail. The ORA- error printed by each statement is the expected result and proves that the rule is enforced. The constraints were created without explicit names, so Oracle reports system-generated names of the form `SYS_C00xxxx`, prefixed with the schema owner. The console block below shows the output of each test.

```
SQL> INSERT INTO purchase_order VALUES (9, 99, 1, 5, SYSDATE);
INSERT INTO purchase_order VALUES (9, 99, 1, 5, SYSDATE)
*
ERROR at line 1:
ORA-02291: integrity constraint (SYSTEM.SYS_C004354) violated - parent key not
found


SQL> INSERT INTO product VALUES (9, 'Test', -5, 1);
INSERT INTO product VALUES (9, 'Test', -5, 1)
*
ERROR at line 1:
ORA-02290: check constraint (SYSTEM.SYS_C004345) violated


SQL> INSERT INTO purchase_order VALUES (9, 1, 1, 0, SYSDATE);
INSERT INTO purchase_order VALUES (9, 1, 1, 0, SYSDATE)
*
ERROR at line 1:
ORA-02290: check constraint (SYSTEM.SYS_C004352) violated


SQL> INSERT INTO shipment VALUES (9, 1, SYSDATE, 'Lost');
INSERT INTO shipment VALUES (9, 1, SYSDATE, 'Lost')
*
ERROR at line 1:
ORA-02290: check constraint (SYSTEM.SYS_C004357) violated


SQL> DELETE FROM supplier WHERE supplier_id = 1;
DELETE FROM supplier WHERE supplier_id = 1
*
ERROR at line 1:
ORA-02292: integrity constraint (SYSTEM.SYS_C004354) violated - child record
found


SQL> INSERT INTO supplier (supplier_id, supplier_name) VALUES (1, 'Dup');
INSERT INTO supplier (supplier_id, supplier_name) VALUES (1, 'Dup')
*
ERROR at line 1:
ORA-00001: unique constraint (SYSTEM.SYS_C004342) violated
```

The same foreign-key constraint (`SYS_C004354`, from Purchase_Order to Supplier) is reported by both T9 and T13: once because the parent key is missing, and once because a child record exists.

16 Reports

The Supply Chain Management Database generates reports to provide useful information about order values, low stock and the orders placed with suppliers. These reports help the manager and the purchase officer monitor the activities of the system and make better decisions.

16.1 Order Value Report

The order value report gives the total value of each purchase order using the function `get_order_value`. Order 1 is the largest because it is for 20 microcontroller boards at the current price of 900 ($20 \times 900 = 18{,}000$).

```
SELECT order_id, get_order_value(order_id) AS order_value
FROM purchase_order;
```

Table 9: Sample Order Value Report

order_id	order_value
1	18,000
2	3,500
3	4,200

16.2 Low-Stock Report

The low-stock report lists the products whose stock is below 10 units and which therefore need replenishment: the microcontroller board (8 units) and the battery (4 units). It is produced by the cursor of Section 12.2.

Table 10: Sample Low-Stock Report

product_name	stock_quantity
Microcontroller Board	8
Battery	4

16.3 Order Details Report

The order details report shows, for each order, the supplier, the product and the quantity ordered, using the three-table join of Section 10.2.

Table 11: Sample Order Details Report

order_id	supplier_name	product_name	quantity
1	Apex Components	Microcontroller Board	20
2	Nova Packaging	Protective Carton	100
3	Apex Components	Battery	10

17 Results and Discussion

The schema was created without errors and enforces integrity at three levels. At the column level, NOT NULL, CHECK and DEFAULT constraints reject invalid values such as a negative price, an order quantity of zero or an unknown shipment status, and supply sensible defaults for stock and shipment status. At the table level, primary and foreign keys preserve entity and referential integrity, as shown by the parent-key, child-record and duplicate-key errors in tests T9, T13 and T14. At the business-rule level, triggers supply the order date automatically (order 4 received the date of the session) and update the stock when goods are delivered: after shipment 1 was marked as delivered, the stock of the microcontroller board rose from 8 to 28, exactly the 20 units of the order.

The queries confirm that the relational design supports the reporting requirements of a supply chain. The join combines orders with supplier and product names; the function reports order values (order 1 is the largest at 18,000, since its price was updated to 900, while orders 2 and 3 are worth 3,500 and 4,200); and the cursor identifies the two items, the microcontroller board (8 units) and the battery (4 units), that need replenishment. Because the unit price is stored only in the Product table, a price change is reflected immediately in every order value; the drawback is that the price actually agreed for an old order is not preserved.

The transaction demonstrations show the ACID properties concretely: a partial ROLLBACK TO SAVEPOINT undoes only the later statement (one temporary supplier remained pending), and COMMIT makes changes durable. GRANT and REVOKE show that read access to the Product data can be given and withdrawn. Together these features illustrate why a DBMS is preferable to ad-hoc file handling for supply chain data.

18 Conclusion

This project designed and implemented a Supply Chain Management Database on Oracle. Starting from an ER model with four entities, a relational schema normalized to BCNF was derived and realized with integrity constraints. The implementation exercises the main DBMS topics: DDL with ALTER TABLE, DML, a join across three tables, a stored function, a cursor, triggers for defaulting the order date and updating stock, transactions with SAVEPOINT, COMMIT and ROLLBACK, and access control with GRANT and REVOKE. The result is a consistent and easily queried database that clearly outperforms manual or spreadsheet-based record keeping, while remaining simple enough to explain and defend in a viva voce. Future enhancements include a graphical front end, role-based security, warehouse-wise inventory and delivery records.

19 Mapping POs and SDGs

19.1 Program Outcomes (POs) Mapping

Table 12: Mapping of Program Outcomes (POs) to Project Relevance

PO No.	**Program Outcome**	**Relevance**
PO1	Engineering Knowledge	Applied relational model, normalization and SQL/PL/SQL fundamentals.
PO2	Problem Analysis	Analysed the redundancy and inconsistency of manual supply chain records.
PO3	Design/Development of Solutions	Designed an ER model and a BCNF schema; implemented it in Oracle.
PO4	Conduct Investigations	Verified the behaviour of constraints, triggers, function and transactions with outputs captured in SQL*Plus.
PO5	Modern Tool Usage	Used Oracle Database, SQL*Plus, ERDPlus and LaTeX/Overleaf.
PO6	Engineer and Society	Provides a reliable, reusable record-keeping system for small businesses and institutions.

Table 13: Mapping of Sustainable Development Goals (SDGs) to Project Relevance

SDG No.	**Goal**	**Relevance**
SDG 4	Quality Education	Serves as a complete, reproducible DBMS learning artifact.
SDG 8	Decent Work and Economic Growth	Supports efficient, well-documented purchasing and logistics operations.
SDG 9	Industry, Innovation and Infrastructure	Demonstrates structured data management for institutions and businesses.
SDG 12	Responsible Consumption and Production	Better stock visibility (low-stock report) helps avoid over-ordering, and replaces paper registers and spreadsheets.

20 Future Scope

The current system is a back-end database exercised through SQL*Plus; it has no graphical front end and no role-based access control beyond the GRANT and REVOKE demonstration. In the present design a purchase order is for a single product, the price agreed for an order is not stored with the order, and the stock is increased only when an existing shipment is updated to `Delivered` (a shipment inserted directly with the status `Delivered` does not change the stock). Future work could add a separate order-item table so that an order can contain many products, store the agreed price with each order, add warehouses with warehouse-wise inventory and stock transfers, record deliveries, carriers and tracking numbers, define views and indexes, add database users and privileges for purchase officers, warehouse operators and managers, generate reorder suggestions automatically, and use explicit row locking to demonstrate concurrency control.

21 Bibliography

References

[1] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson Education, 2016.

[2] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. McGraw-Hill, 2019.

[3] R. Ramakrishnan and J. Gehrke, *Database Management Systems*, 3rd ed. McGraw-Hill, 2003.

[4] C. J. Date, *An Introduction to Database Systems*, 8th ed. Pearson Education, 2004.

[5] Oracle Corporation, *Oracle Database SQL Language Reference*. Oracle Help Center, `https://docs.oracle.com`.

[6] Oracle Corporation, *Oracle Database PL/SQL Language Reference*. Oracle Help Center, `https://docs.oracle.com`.

[7] E. F. Codd, "A relational model of data for large shared data banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970.

[8] P. P. Chen, "The entity-relationship model — toward a unified view of data," *ACM Transactions on Database Systems*, vol. 1, no. 1, pp. 9–36, 1976.

[9] S. Chopra and P. Meindl, *Supply Chain Management: Strategy, Planning, and Operation*, 6th ed. Pearson Education, 2016.